



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 1

1. Reading a CSV File

Scenario: You received a sales.csv file from your manager.

Question: How do you load it into Pandas?

Answer: `import pandas as pd`
`df = pd.read_csv("sales.csv")`

Explanation:

- The `read_csv()` function reads data from a CSV file.
- It returns a DataFrame, which is a 2-dimensional table with rows and columns.

Why Pandas?

- ★ Easy to use and beginner-friendly
- ★ Powerful data analysis tools
- ★ Handles missing data easily
- ★ Built-in functions for data cleaning
- ★ Great for data manipulation and exploration

2. View First Records

Scenario: You want to check whether the data loaded correctly.

Question: How do you see the first 5 rows?

Answer: `df.head()`

Explanation:

- The `head()` function shows the first 5 rows by default.
- You can pass a number inside `head(n)` to see first n rows.
For example: `df.head(10)` will show first 10 rows.

Index	<code>df.head()</code>	<code>df.head(10)</code>
0	Row 1	Row 1 to 10
1	Row 2	Row 1 to 10
2	Row 3	Row 1 to 10
3	Row 4	Row 1 to 10
4	Row 5	Row 1 to 10

3. View Last Records

Scenario: You want to check the last records.

Question: How do you see the last 5 rows?

Answer: `df.tail()`

Explanation:

- The `tail()` function shows the last 5 rows by default.
- You can pass a number inside `tail(n)` to see last n rows.
For example: `df.tail(10)` will show last 10 rows.

Common Pandas Functions

- `+` : addition
- `-` : subtraction
- `*` : multiplication
- `/` : division
- `**` : power
- `%` : modulo



4. Find Number of Rows and Columns

Scenario: Your manager asks how big the dataset is.

Question: How do you find the number of rows and columns?

Answer: `df.shape`

Explanation: → The `shape` attribute returns a tuple (rows, columns).
→ For example, if output is (100, 5), it means the dataset has 100 rows and 5 columns.

```
import pandas as pd
df = pd.read_csv("sales.csv")
print(df.shape) # Output: (100, 5)
```

5. List All Columns

Scenario: You forgot the column names.

Question: How do you list all the columns in the dataset?

Answer: `df.columns`

Explanation: → The `columns` attribute returns an Index object containing all column names.
→ It helps you quickly understand what fields are available in the dataset.

```
import pandas as pd
df = pd.read_csv("sales.csv")
print(df.columns)
# Output: Index(['Date', 'Product', 'Sales', 'Region'],
              dtype='object')
```




50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 2

6 Check Data Types

Scenario: Your manager wants to know the data type of each column to understand the dataset better.

Question: How do you check the data types of all columns?

Answer: `df.dtypes`

Explanation:

- The `dtypes` attribute returns the data type of each column in the DataFrame.
- It helps identify whether a column is integer, float, object (string), boolean or datetime.
- This is useful before doing calculations or conversions.

7 Get Dataset Summary (Info)

Scenario: You want a quick summary of the dataset including missing values.

Question: How do you get a concise summary of the DataFrame?

Answer: `df.info()`

Explanation: The `info()` function prints a summary that includes:

- Total number of rows and columns
- Column names
- Non-null count (how many non-missing values)
- Data types of each column
- Memory usage
- It is very helpful for an overview of the dataset.

8 Get Statistical Summary

Scenario: Your manager needs min, max, mean, and other statistics of numeric columns.

Question: How do you get statistical summary of the dataset?

Answer: `df.describe()`

Explanation: The `describe()` function returns key statistics for numeric columns:

- count, mean, std (standard deviation), min, 25%, 50%, 75%, max.
- It helps in understanding the distribution of data.

9 Select One Column

Scenario: You only need the "Employee_Name" column for a report.

Question: How do you select a single column from the DataFrame?

Answer: `df["Employee_Name"]`

Explanation: This returns the specified column as a Series.

- A Series is a one-dimensional labeled array.

10 Select Multiple Columns

Scenario: You need the "Employee_Name" and "Salary" columns for analysis.

Question: How do you select multiple columns from the DataFrame?

Answer: `df[["Employee_Name", "Salary"]]`

Explanation: This returns a new DataFrame with only the selected columns.

- We use double brackets `[[]]` to select multiple columns.
- The result is still a DataFrame.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 3

11 Find Missing Values

Scenario: Some employees did not provide their email or phone number.

Question: How do you find missing values in each column?

Answer: `df.isnull().sum()`

Explanation: The `isnull()` function returns a DataFrame of True/False indicating missing values. The `sum()` function then counts the total number of missing values in each column.

→ This helps to identify which columns have missing data and how much is missing.

12 Remove Missing Values

Scenario: You want to remove all rows that have any missing data.

Question: How do you drop rows with missing values?

Answer: `df.dropna()`

Explanation: The `dropna()` function removes rows that contain NaN (missing) values.

→ By default, it removes rows where any column has a missing value (`how = 'any'`).

→ This is useful when you want only complete records for further analysis.

13 Replace Missing Values

Scenario: For numerical columns, replace missing values with 0.

Question: How do you fill missing values with a specific value?

Answer: `df.fillna(0)`

Explanation: The `fillna(value)` function replaces all NaN values with the specified value.

→ Here, 0 is used, so all missing values are replaced by 0.

→ This is helpful when missing values can be treated as zero (e.g., Salary, Bonus, etc.).

14 Rename Column

Scenario: The column name "Emp_Name" is not clear. You want to rename it to "Employee_Name".

Question: How do you rename a column?

Answer: `df.rename(columns = {"Emp_Name": "Employee_Name"}, inplace = True)`

Explanation: The `rename()` function changes column names.

→ `columns = {"old_name": "new_name"}` is used to specify the change.

→ `inplace = True` applies the change to the original DataFrame.

15 Add New Column

Scenario: You want to add a new column called "Bonus" which is 10% of the employee's Salary.

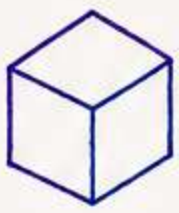
Question: How do you create a new column based on existing data?

Answer: `df["Bonus"] = df["Salary"] * 0.10`

Explanation: This creates a new column "Bonus".

→ For each row, Salary is multiplied by 10% (0.10) and stored in the new column.

→ This does not change the original data.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 4

16 Delete Column

Scenario: The column "Temp" is no longer required in the dataset.

Question: How do you delete the "Temp" column from the DataFrame?

Answer: `df.drop('Temp', axis=1, inplace=True)`

Explanation: The `drop()` function removes a row or column.

→ `axis=1` specifies that we want to drop a column (`axis=0` is for rows).

→ `inplace=True` makes the change permanent in the original DataFrame.

→ If `inplace` is not used, a new DataFrame is returned.

17 Filter Rows with Condition

Scenario: You want to find all employees whose salary is greater than 50,000.

Question: How do you filter these records?

Answer: `df[df["Salary"] > 50000]`

Explanation: We use square brackets `[]` to filter rows based on a condition.

→ `df["Salary"] > 50000` returns a Boolean Series (True/False).

→ Only rows where the condition is True are returned.

18 Filter Rows with Multiple Conditions

Scenario: Find employees from "IT" department who have salary greater than 50,000 and age less than 30.

Question: How do you filter these records?

Answer: `df[(df["Department"] == "IT") & (df["Salary"] > 50000) & (df["Age"] < 30)]`

Explanation: We combine multiple conditions using the `&` operator.

→ Each condition is written inside parentheses `()`.

→ All conditions must be True for a row to be selected.

19 Sort Data

Scenario: You want to display the employees in descending order of salary.

Question: How do you sort the DataFrame by "Salary" in descending order?

Answer: `df.sort_values('Salary', ascending=False)`

Explanation: The `sort_values()` function sorts the DataFrame by the given column.

→ `ascending=False` means highest values will appear first.

→ By default, `ascending=True` (smallest first).

20 Get Top N Records

Scenario: Find the top 5 highest paid employees.

Question: How do you get the top 5 records based on salary?

Answer: `df.nlargest(5, 'Salary')`

Explanation: The `nlargest(n, column)` function returns the top 'n' rows with the largest values in the specified column.

→ It is more efficient than sorting the entire DataFrame when you only need top N records.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 5

21 Count Unique Values in a Column

Scenario: You want to know how many unique departments are there.

Question: How do you count unique departments in the dataset?

Answer: `df["Department"].nunique()`

Explanation: The `nunique()` function returns the count of unique (distinct) values in a column.

→ It ignores duplicate values.

→ This helps in understanding the variety in the data.

22 Find Unique Values in a Column

Scenario: You want to see all the departments available in the company.

Question: How do you get the unique values from the "Department" column?

Answer: `df["Department"].unique()`

Explanation: The `unique()` function returns an array of unique values present in the specified column.

→ It shows which departments exist in the dataset.

→ Useful for data exploration and validation.

23 Count Values in Each Category

Scenario: You want to know how many employees are in each department.

Question: How do you count the number of employees in each department?

Answer: `df["Department"].value_counts()`

Explanation: The `value_counts()` function returns a Series containing the count of each unique value in descending order.

→ It shows how many times each department appears.

→ Helps in analyzing the distribution of employees.

24 Replace Values in a Column

Scenario: In the "Gender" column, 'M' and 'F' need to be replaced with 'Male' and 'Female'.

Question: How do you replace values in a column?

Answer: `df["Gender"] = df["Gender"].replace({"M": "Male", "F": "Female"})`

Explanation: The `replace()` function replaces old values with new values.

→ It can handle one or multiple replacements using a dictionary.

→ It does not change other values.

25 Map Values in a Column

Scenario: You want to map department codes to full department names.

Codes: IT → Information Technology, HR → Human Resources.

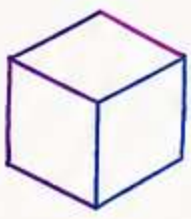
Question: How do you map values in a column?

Answer: `df["Dept_Name"] = df["Dept_Code"].map({"IT": "Information Technology", "HR": "Human Resources"})`

Explanation: The `map()` function substitutes values based on a given dictionary.

→ It is used for one-to-one mapping.

→ If a value is not found in the dictionary, it becomes NaN.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 6

26 Find Maximum Salary

Scenario: Your manager wants to know the highest salary in the company.

Question: How do you find the maximum salary from the DataFrame?

Answer: `df["Salary"].max()`

Explanation: The `max()` function returns the largest value in the specified column.

→ It scans all the values in 'Salary' and picks the highest one.

→ Useful for quick insights like top salary offered.

27 Find Minimum Salary

Scenario: You need to know the lowest salary paid to any employee.

Question: How do you find the minimum salary?

Answer: `df["Salary"].min()`

Explanation: The `min()` function returns the smallest value in the column.

→ It helps identify the lowest-paid employee.

→ Useful in salary analysis and fairness checks.

28 Calculate Average Salary by Department

Scenario: You want to know the average salary for each department.

Question: How do you calculate the average salary department-wise?

Answer: `df.groupby('Department')['Salary'].mean()`

Explanation: We use `groupby()` to group rows by 'Department' and then apply `mean()` on 'Salary'.

→ It returns the average salary for each department.

→ Very useful for department performance comparison.

29 Sum of Salaries by Department

Scenario: Find the total salary expense for each department.

Question: How do you calculate the total salary paid in each department?

Answer: `df.groupby('Department')['Salary'].sum()`

Explanation: The `sum()` function adds all salary values within each department.

→ It gives total payroll cost per department.

→ Helpful for budgeting and expense tracking.

30 Count Employees by Department

Scenario: You want to know how many employees work in each department.

Question: How do you count employees in each department?

Answer: `df['Department'].value_counts()`
or
`df.groupby('Department').size()`

Explanation: `value_counts()` counts how many times each department appears.

→ `groupby().size()` gives the count of rows in each group.

→ Both methods return the number of employees in each department.

→ Useful for workforce distribution analysis.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 7

31 Reset Index

Scenario: After deleting some rows, the index is no longer in order.

Question: How do you reset the index of the DataFrame?

Answer: `df.reset_index(drop=True)`

Explanation: The `reset_index()` function resets the index to the default (0, 1, 2, 3, ...) and removes the old index.

→ `drop=True` removes the old index column.

→ Very useful after filtering or dropping rows.

32 Convert Data Type

Scenario: The "Age" column is stored as text (object) but it should be integer.

Question: How do you convert the "Age" column to integer?

Answer: `df["Age"] = df["Age"].astype(int)`

Explanation: The `astype()` method changes the data type of a column.

→ Here, we convert all values in "Age" to integer.

→ This helps in doing numeric operations and calculations.

33 Read Excel File

Scenario: You received an Excel file named "employees.xlsx".

Question: How do you read it using Pandas?

Answer: `df = pd.read_excel("employees.xlsx")`

Explanation: The `read_excel()` function reads an Excel file and returns a DataFrame.

→ Requires `openpyxl` library (`pip install openpyxl`).

→ You can also specify sheet name using `sheet_name="Sheet1"`.

34 Save Data to CSV File

Scenario: After cleaning the data, you want to save it as a CSV file.

Question: How do you export the DataFrame to a CSV file?

Answer: `df.to_csv("output.csv", index=False)`

Explanation: The `to_csv()` function writes the DataFrame to a CSV file.

→ `index=False` prevents writing row index in the file.

→ The file can be opened in Excel or any text editor.

35 Save Data to Excel File

Scenario: You want to save the cleaned data into an Excel file.

Question: How do you export the DataFrame to an Excel file?

Answer: `df.to_excel("output.xlsx", index=False)`

Explanation: The `to_excel()` function writes the DataFrame to an Excel file.

→ `index=False` prevents index column from being written.

→ Requires `openpyxl` or `xlsxwriter` library.

→ Useful for sharing reports in Excel format.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 8

36 Group By Department and Calculate Average Salary

Scenario: You want to know the average salary for each department.
Question: How do you get the average salary department-wise?
Answer: `df.groupby('Department')['Salary'].mean()`
Explanation: The `groupby()` function groups the data by 'Department'.
→ Then, `mean()` calculates the average of the 'Salary' column for each group.
→ Returns a Series with department names as index and average salary as values.

37 Find Number of Employees in Each Department

Scenario: Your manager wants to know how many employees are in each department.
Question: How do you count employees in each department?
Answer: `df['Department'].value_counts()`
Explanation: The `value_counts()` function counts unique values in the 'Department' column.
→ Returns department names as index and counts as values.
→ Helpful for quick headcount reports.

38 Sort Data by Department (A to Z) and Salary (High to Low)

Scenario: You want to view employees sorted by department name and within each department by highest salary.
Question: How do you sort the DataFrame?
Answer: `df.sort_values(['Department', 'Salary'], ascending=[True, False])`
Explanation: We sort first by 'Department' in ascending order (A to Z).
→ For 'Salary', we use False to get highest salary first.
→ Multiple sorting helps in organized reporting.

39 Select Top 10 Highest Paid Employees

Scenario: You need a list of top 10 employees based on salary.
Question: How do you select top 10 highest salary records?
Answer: `df.nlargest(10, 'Salary')`
Explanation: `nlargest(n, column)` returns the first n largest values from the specified column.
→ Returns the top 10 rows with highest salary.
→ More efficient than sorting the whole DataFrame.

40 Select Bottom 10 Lowest Paid Employees

Scenario: You need a list of bottom 10 employees based on salary.
Question: How do you select bottom 10 lowest salary records?
Answer: `df.nsmallest(10, 'Salary')`
Explanation: `nsmallest(n, column)` returns the first n smallest values from the specified column.
→ Returns the 10 rows with lowest salary.
→ Useful for identifying underpaid employees or interns.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 9

41 Remove Duplicate Rows from DataFrame

Scenario: Your dataset contains duplicate rows and you need unique records.

Question: How do you remove duplicate rows in Pandas?

Answer: `df.drop_duplicates()`

Explanation: The `drop_duplicates()` function removes duplicate rows.

→ By default, it considers all columns to identify duplicates.

→ Keeps the first occurrence and removes the rest.

→ You can specify subset of columns and `keep = 'first'` or `'last'`.

Example: `df.drop_duplicates(subset=['Email'], keep='last')`

This is very useful for cleaning real-world data.

42 Handle Missing Values by Filling with Mean

Scenario: Some numeric columns have missing values and you want to fill them with the average of that column.

Question: How do you fill missing values in a column with its mean?

Answer: `df['Column'].fillna(df['Column'].mean(), inplace=True)`

Explanation: The `fillna()` function replaces NaN values with a specified value.

→ We use `mean()` for numeric columns.

→ `inplace=True` updates the DataFrame without creating a new copy.

→ This helps maintain data consistency.

43 Replace Missing Values with Forward Fill

Scenario: You have time-series data with missing values and want to fill them with the previous valid value.

Question: How do you forward fill missing values?

Answer: `df.ffill(inplace=True)`

Explanation: The `ffill()` function propagates the last valid observation forward.

→ Ideal for time-series data where next values depend on previous ones.

→ You can also specify `limit` to fill only a certain number of rows.

→ Backward fill (`bfill`) can be used to use next valid value.

44 Rename Columns in DataFrame

Scenario: Your DataFrame has old column names and you want to rename them to new meaningful names.

Question: How do you rename one or more columns?

Answer: `df.rename(columns={'old_name': 'new_name'}, inplace=True)`

Explanation: The `rename()` function changes column names.

→ Use a dictionary to map old names to new names.

→ `inplace=True` modifies the original DataFrame.

→ Helps in better readability and understanding of data.

45 Create a New Column Based on Existing Columns

Scenario: You want to create a new column "Total_Salary" by adding "Basic_Salary" and "Bonus" columns.

Question: How do you create a new column in Pandas?

Answer: `df['Total_Salary'] = df['Basic_Salary'] + df['Bonus']`

Explanation: We can create new columns using arithmetic operations.

→ Pandas performs element-wise operations on columns.

→ The new column is added to the DataFrame.

→ Useful for feature engineering and data analysis.



50 Pandas Fresher-Level Scenario-Based Interview Questions



Page 10

46 Filter Data Between Two Dates

Scenario: You have sales data with a 'Date' column and you want data between 1st Jan 2024 and 31st Jan 2024.

Question: How do you filter the DataFrame for that date range?

Answer: `df[(df['Date'] >= '2024-01-01') & (df['Date'] <= '2024-01-31')]`

Explanation: We use boolean indexing to filter rows where 'Date' falls within the given range.

- Comparisons are inclusive ($\geq$ and $\leq$).
- Make sure 'Date' column is in datetime format (use `pd.to_datetime()`).
- Very useful for reports and dashboards.

47 Add a Percentage Column

Scenario: You have 'Sales' and 'Target' columns and want to add a column 'Achievement%' = $(\text{Sales} / \text{Target}) * 100$.

Question: How do you create this new column?

Answer: `df['Achievement%'] = (df['Sales'] / df['Target']) * 100`

Explanation: We perform element-wise division and multiply by 100.

- Handle division by zero using `replace` or `where` if needed.
- Round the result using `.round(2)` if required.
- Useful for KPI and performance metrics.

48 Pivot Data for Summary Report

Scenario: You have sales data with columns: 'Date', 'Product', 'Region', 'Sales'. You want total sales of each product in each region.

Question: How do you create a pivot table for this?

Answer: `df.pivot_table(values='Sales', index='Product', columns='Region', aggfunc='sum')`

Explanation: `pivot_table()` reshapes data and aggregates values.

- index becomes rows, columns becomes columns of the table.
- `aggfunc` defines the aggregation (sum, mean, count, etc.).
- Great for creating cross-tab reports.

49 Find Duplicate Records

Scenario: Your dataset may contain duplicate rows and you want to identify them.

Question: How do you find duplicate rows in a DataFrame?

Answer: `df[df.duplicated(keep=False)]`

Explanation: `duplicated()` marks duplicate rows.

- `keep=False` shows all duplicates (first + repeats).
- `keep='first'` (default) shows only duplicate rows except first.
- Useful for data quality checks.

50 Export Filtered Data to Excel

Scenario: After filtering your data, you want to export it to an Excel file.

Question: How do you save the filtered DataFrame to Excel?

Answer: `filtered_df.to_excel('filtered_data.xlsx', index=False, sheet_name='Report')`

Explanation: `to_excel()` writes the DataFrame to an Excel file.

- `index=False` prevents writing row index.
- `sheet_name` names the worksheet.
- Essential for sharing outputs with stakeholders.

★ These 50 **scenario-based** questions cover **real-world use cases** that recruiters love in **pandas**! Perfect for fresher-level **data analyst** interviews!

